



ChefStack

Guía técnica del frontend

Deep-dive de la SPA en Vue 3 + Vite: estructura, vistas, componentes, composables, stores, capa API, router y sistema de diseño.

1 Stack y arranque

El frontend de ChefStack es una **SPA (Single Page Application)** construida con Vue 3 usando la Composition API y la sintaxis `<script setup>`. El empaquetado y el servidor de desarrollo corren sobre Vite. La aplicación se despliega en Vercel y consume una API REST publicada en Railway.

Tecnología	Versión / detalle	Rol en el frontend
Vue 3	Composition API · <code><script setup></code>	Framework de UI reactiva
Vite	Bundler + dev server	Build, HMR, variables <code>VITE_*</code>
Pinia	Store global	Estado de sesión (<code>stores/auth.js</code>)
Vue Router	v4	Enrutado SPA + guards de acceso
Axios	Cliente HTTP	Instancia con interceptor de token
Firebase SDK	v10	Autenticación (email/contraseña + Google)

Entornos en vivo

Recurso	URL
Frontend (Vercel)	<code>https://chefstack-web.vercel.app</code>
API (Railway)	<code>https://chefstack-api-production.up.railway.app</code>
Variable de entorno	<code>VITE_API_URL</code> apunta a la API; fallback <code>http://localhost:8080</code>

Cómo arranca `main.js`

El orden del arranque es deliberado: la app se monta **solo después** de inicializar la autenticación, para que el router conozca el estado de sesión antes de evaluar los guards.

```
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import router from './router'
import { useAuthStore } from './stores/auth'
import './assets/styles.css'

const app = createApp(App)
app.use(createPinia())

const auth = useAuthStore()
auth.inicializar() // dispara el observador de sesión de Firebase

app.use(router)
app.mount('#app')
```

Clave: `auth.inicializar()` no bloquea el montaje. El guard del router espera por la promesa `auth.listo` (ver sección 4), de modo que la UI aparece de inmediato y la resolución de sesión ocurre en paralelo.

Layout raíz — `App.vue`

El componente raíz compone el esqueleto persistente de la aplicación:

```

App.vue
├─ <NavBar/>      barra superior (menú según sesión)
├─ <RouterView/>  vista activa con <transition> de entrada
│   └─ <AppFooter/>  pie editorial
├─ <ToastHost/>    capa de notificaciones (useToast)
└─ <ConfirmHost/>  modal de confirmación (useConfirm)
```

2 Estructura de carpetas

```
chefstack-web/
├─ package.json          # dependencias y scripts (dev/build/preview)
├─ vite.config.js        # config de Vite (alias, plugin Vue)
├─ index.html            # punto de montaje (#app)
├─ vercel.json           # rewrites SPA para Vercel
├─ .env.example          # plantilla de variables VITE_*
├─ public/
│   └─ favicon.svg
└─ src/
    ├─ main.js            # arranque: Pinia, router, init auth
    ├─ App.vue            # layout: NavBar + RouterView + Footer + Toast + Confirm
    ├─ assets/
    │   └─ styles.css     # sistema de diseño / tokens CSS
    ├─ api/
    │   ├─ http.js        # axios + interceptor de token
    │   ├─ recetas.js     # CRUD de recetas
    │   ├─ categorias.js  # CRUD de categorías
    │   ├─ favoritos.js   # favoritos del usuario
    │   └─ uploads.js     # subida de imágenes (FormData)
    ├─ firebase/
    │   └─ index.js       # init Firebase + authService (fachada)
    ├─ stores/
    │   └─ auth.js        # Pinia: sesión + promesa "listo"
    ├─ router/
    │   └─ index.js       # rutas + guards de acceso
    ├─ composables/
    │   ├─ useToast.js    # notificaciones (singleton)
    │   └─ useConfirm.js  # confirmación que devuelve Promise<boolean>
    ├─ utils/
    │   └─ formato.js     # dificultadInfo, imagenSrc, IMAGEN_POR_DEFECTO
    ├─ components/
    │   ├─ NavBar.vue
    │   ├─ RecetaCard.vue
    │   ├─ ConfirmHost.vue
    │   ├─ ToastHost.vue
    │   └─ AppFooter.vue
    └─ views/
        ├─ HomeView.vue
        ├─ RecetaDetalleView.vue
        ├─ RecetaFormView.vue
        ├─ CategoriasView.vue
        ├─ FavoritosView.vue
        ├─ LoginView.vue
        └─ RegistroView.vue
```

3 Capa de datos (API)

Toda la comunicación con el backend pasa por una única instancia de Axios definida en `api/http.js`. Esa instancia centraliza la `baseUrl` y, sobre todo, inyecta el token de Firebase en cada petición mediante un **interceptor de request**.

`api/http.js` — instancia + interceptor

```
import axios from 'axios'
import { authService } from '../firebase'

const http = axios.create({
  baseUrl: import.meta.env.VITE_API_URL || 'http://localhost:8080'
})

// Adjunta el idToken de Firebase a cada petición si hay sesión
http.interceptors.request.use(async (config) => {
  const usuario = authService.usuarioActual()
  if (usuario) {
    const token = await usuario.getIdToken()
    config.headers.Authorization = `Bearer ${token}`
  }
  return config
})

export default http
```

El token se obtiene con `getIdToken()` en cada request, de modo que Firebase entrega siempre un token vigente (se refresca solo). El backend valida ese Bearer para autorizar operaciones protegidas (crear/editar/eliminar, favoritos).

Módulos por recurso

Cada recurso tiene su módulo que envuelve a `http` y expone funciones con nombres en español, devolviendo directamente `response.data`.

Módulo	Funciones expuestas	Endpoint base
recetas.js	listar(params), obtener(id), crear(datos), actualizar(id, datos), eliminar(id)	/api/recetas
categorias.js	listar(), crear(datos), actualizar(id, datos), eliminar(id)	/api/categorias
favoritos.js	listar(), agregar(recetaId), quitar(recetaId)	/api/favoritos
uploads.js	subir(archivo) → arma FormData y hace POST multipart; devuelve la URL de la imagen	/api/uploads

```
// api/uploads.js (esencia)
export async function subir(archivo) {
  const form = new FormData()
  form.append('file', archivo)
  const { data } = await http.post('/api/uploads', form)
  return data // { url: '/api/uploads/123' }
}
```

4 Autenticación en el front

La autenticación se apoya en Firebase Auth, pero el resto de la app **nunca** habla con Firebase directamente: lo hace a través de la fachada `authService` y del store de Pinia. Esto desacopla la app del SDK y permite un único punto de control.

firebase/index.js — init + authService

Inicializa Firebase con una config pública (que puede sobrescribirse con variables `VITE_*`) y expone una fachada de métodos en español:

Método de <code>authService</code>	Responsabilidad
registrar(email, pass)	createUserWithEmailAndPassword
ingresar(email, pass)	signInWithEmailAndPassword
ingresarConGoogle()	signInWithPopup con <code>GoogleAuthProvider</code>
salir()	signOut
observarSesion(cb)	onAuthStateChanged — notifica cambios de sesión
usuarioActual()	devuelve <code>auth.currentUser</code> (usado por el interceptor)

stores/auth.js — la promesa `listo` y su timeout

El store de Pinia mantiene el estado reactivo de la sesión y resuelve un problema clásico de las apps con auth asíncrona: *¿qué muestro mientras Firebase decide si hay sesión?*

Miembro	Tipo	Descripción
<code>usuario</code>	state	objeto de usuario o <code>null</code>
<code>cargando</code>	state	<code>true</code> hasta conocer el estado inicial
<code>autenticado</code>	computed	<code>!!usuario</code>
<code>nombre</code>	computed	displayName o email del usuario
<code>listo</code>	Promise	se resuelve al conocer la sesión o al vencer el timeout
<code>inicializar()</code>	acción	arranca <code>observarSesion</code> ; es a prueba de fallos

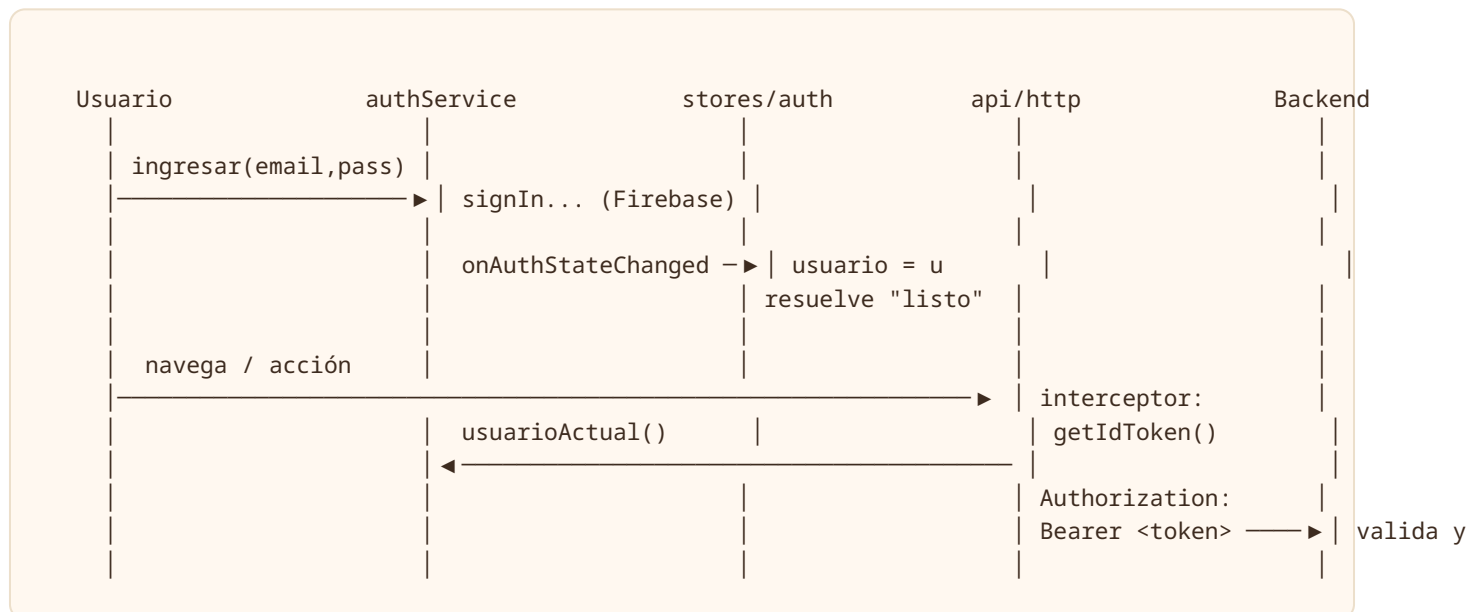
```
// stores/auth.js (esencia de la promesa "listo")
let resolverListo
const listo = new Promise((resolve) => { resolverListo = resolve })

function inicializar() {
  // Red de seguridad: si Firebase tarda, no dejamos la app en blanco
  const timeout = setTimeout(() => resolverListo(), 2000)

  authService.observarSesion((u) => {
    usuario.value = u
    cargando.value = false
    clearTimeout(timeout)
    resolverListo() // idempotente: resolver dos veces no afecta
  })
}
```

Por qué el timeout de 2 s: si la red de Firebase falla o tarda, el guard del router quedaría esperando para siempre y la app se vería en blanco. El timeout garantiza que `auth.listo` siempre se resuelva, tratando al usuario como invitado en el peor caso. Por eso `inicializar()` es "a prueba de fallos".

Flujo login → token → petición



5 Rutas y navegación

El router (`router/index.js`) define las rutas de la SPA con metadatos de acceso y un único **guard global** `beforeEach` .

Ruta	Vista	Acceso
<code>/</code>	HomeView	Público
<code>/recetas/:id</code>	RecetaDetalleView	Público
<code>/recetas/nueva</code>	RecetaFormView	Privado (<code>requiereAuth</code>)
<code>/recetas/:id/editar</code>	RecetaFormView	Privado (<code>requiereAuth</code>)
<code>/categorias</code>	CategoriasView	Privado (<code>requiereAuth</code>)
<code>/favoritos</code>	FavoritosView	Privado (<code>requiereAuth</code>)
<code>/login</code>	LoginView	Solo invitados (<code>soloInvitados</code>)
<code>/registro</code>	RegistroView	Solo invitados (<code>soloInvitados</code>)

El guard global

Antes de cada navegación, el guard **espera a que se conozca el estado de sesión** y luego aplica las reglas de los metadatos:


```
router.beforeEach(async (to) => {
  const auth = useAuthStore()
  await auth.listo // sincroniza con Firebase (o timeout)

  if (to.meta.requiresAuth && !auth.autenticado) {
    return { name: 'login' } // ruta privada sin sesión → login
  }
  if (to.meta.soloInvitados && auth.autenticado) {
    return { name: 'home' } // ya autenticado → fuera del login
  }
  // resto: navegación permitida
})
```

El `await auth.listo` es lo que evita el "parpadeo de autenticación": el guard nunca decide con información incompleta. Combinado con el timeout del store, jamás se cuelga.

6 Vistas y componentes

Vistas (`src/views/`)

HomeView

Pantalla principal. Compone un **hero** editorial, un **buscador con debounce** (evita disparar una petición por cada tecla), **filtros por categoría** en forma de chips, y una **grilla de RecetaCard**. Consume `recetas.listar()` y `categorias.listar()`. Datos clave: término de búsqueda, categoría seleccionada y lista de recetas reactiva.

RecetaDetalleView

Detalle de una receta: imagen principal, datos (tiempo, porciones, kcal, dificultad), lista de **ingredientes** y pasos de **preparación**. Si hay sesión, muestra botones de **favorito**, **editar** y **eliminar**. La eliminación usa `useConfirm` (modal propio) en lugar del `alert/confirm` nativo.

RecetaFormView

Formulario reutilizado para **crear** y **editar** (distingue por la presencia de `:id`). Incluye **subida de imagen** con previsualización (archivo vía `uploads.subir()`) o URL directa, e **ingredientes dinámicos** (añadir / quitar filas). Al guardar llama a `recetas.crear()` o `recetas.actualizar()`.

CategoriasView

CRUD de categorías: formulario de alta/edición + lista con acciones. Ruta privada. Consume el módulo `categorias.js`.

FavoritosView

Grilla con las recetas marcadas como favoritas por el usuario autenticado. Consume `favoritos.listar()` y reutiliza `RecetaCard`.

LoginView / RegistroView

Formularios de email/contraseña con botón de **Google**. Login llama a `authService.ingresar()` / `ingresarConGoogle()`; Registro a `authService.registrar()`. Ambas son rutas `soloInvitados`.

Componentes (`src/components/`)

Componente	Responsabilidad	Datos / métodos clave
<code>NavBar.vue</code>	Barra de navegación responsive; el menú cambia según haya o no sesión	lee <code>auth.autenticado</code> y <code>auth.nombre</code> ; acción <code>salir()</code>
<code>RecetaCard.vue</code>	Tarjeta de receta con imagen, categoría, dificultad, tiempo, kcal y efecto hover	prop <code>receta</code> ; usa <code>imagenSrc</code> y <code>dificultadInfo</code>
<code>ConfirmHost.vue</code>	Modal de confirmación propio, montado en el layout raíz	controlado por el estado de <code>useConfirm</code>
<code>ToastHost.vue</code>	Capa de notificaciones (éxito / error) apiladas	renderiza la lista de <code>useToast</code>
<code>AppFooter.vue</code>	Pie de página editorial	estático

7 Composables y utilidades

`useToast.js`

Estado **singleton** de notificaciones (la lista vive fuera de la función, por lo que se comparte entre todos los componentes). Expone `exito(mensaje)` y `error(mensaje)`; cada toast se añade a la lista y se auto-elimina tras un tiempo. `ToastHost` los renderiza.

```
const toasts = ref([]) // singleton: fuera del composabile

export function useToast() {
  const exito = (msg) => agregar(msg, 'exito')
  const error = (msg) => agregar(msg, 'error')
  return { toasts, exito, error }
}
```

`useConfirm.js`

Modal de confirmación que **devuelve una `Promise<boolean>`**, lo que permite usar `await` en el código que pide confirmación (en vez del bloqueante `window.confirm`). `ConfirmHost` muestra el diálogo y resuelve la promesa al aceptar o cancelar.

```
const ok = await confirmar('¿Eliminar esta receta?')
if (ok) { await recetas.eliminar(id); toast.exito('Receta eliminada') }
```

`utils/formato.js`

Export	Qué hace
<code>dificultadInfo(d)</code>	devuelve etiqueta + clase de color para la dificultad (p. ej. fácil/media/difícil)
<code>imagenSrc(url)</code>	normaliza la URL de imagen (ver detalle abajo)
<code>IMAGEN_POR_DEFECTO</code>	placeholder cuando una receta no tiene imagen

Detalle de `imagenSrc`: si la URL es **relativa** (p. ej. `/api/uploads/123`, como las que devuelve `uploads.subir`) la resuelve contra la `baseURL` del backend; si es **absoluta** (empieza por `http`) la deja intacta; si viene vacía, usa `IMAGEN_POR_DEFECTO`.

```
export function imagenSrc(url) {
  if (!url) return IMAGEN_POR_DEFECTO
  if (url.startsWith('http')) return url // absoluta → tal cual
  const base = import.meta.env.VITE_API_URL || 'http://localhost:8080'
  return base + url // relativa → contra backend
}
```

8 Sistema de diseño

El sistema de diseño vive en `assets/styles.css` y persigue una identidad "**editorial cálido de cocina**", evitando deliberadamente la estética genérica de plantilla. Se apoya en variables CSS para tokens de color, tipografía y sombras.

Tipografías

Fuente	Rol
Fraunces	Display serif — titulares y hero, da el tono editorial
Hanken Grotesk	Cuerpo de texto e interfaz — legibilidad y modernidad

Paleta (variables CSS)

Paleta terracota / crema / carbón / hierba, expuesta como tokens:

Variable	Uso
--terracota	color de acento / marca
--crema	fondo cálido principal
--carbon	texto y contraste oscuro
--hierba	accento secundario (verde)

Principios visuales

- **Textura de grano** sutil sobre los fondos para sensación impresa/artesanal.
- **Sombras cálidas** (tintadas, no grises neutros) en tarjetas y modales.
- Componentes con personalidad: **botones, chips de categoría, tarjetas, toasts**.
- **Animaciones de entrada** suaves (la transición del `RouterView` y el aparecer de tarjetas/toasts).

La meta del sistema de diseño es que ChefStack **no se vea como una plantilla genérica**: la combinación de serif display + grotesque, la paleta cálida de cocina y la textura de grano dan una identidad reconocible y coherente en todas las vistas.